File Structure: Assignment #1

100points

Cairo University, Faculty of
Computers and Information

Important Notes

1. Due date **7/3/2018 -10:30 PM**
2. Discussion of assignment will be in your TA's office hours.
3. Group = 2 students from same lab.
4. Your submission must have a readme file with all your information (ID, name, group)
5. Send one compressed file, name you submission with your Group number.
6. Submit your submission using only Acadox system
7. No late submission
8. Do not include any .exe files in your submission
9. Plagiarism is checked using a professional tool, so do not try to copy from your colleges or from internet. Cheaters will be graded by **–ve points**

Answer the following question using C++.

Q1: Finding all palindromes: **Suppose we have a file that contains some words separated by space, comma or full stop. print all substrings that are palindromes in the file. Write the output to a file and print it also to screen. Example :** **Suppose the file contains hello, All IEEE. then the output should be.**

```
h
e
l
ll
o
a
l
ll
l
E
EE
EEE
```

Q2: Consider we want to store the data of notebooks in a file, every notebook has the following attributes:

Field	Data type	example
ID	Char[21] //max len =20	xa12345
Brand	char [21] //max len = 20	Apple
Processor desc.	char [51] //max len = 50	Intel Core i5=2.7 gh
Operating System	Char [21] //max 20	Mac OS X 10.10

- The file organization method is: **length indicator based record, delimited fields.**
- The above data in the table will be formatted as follows:

51 xa12345|Apple|Intel Core i5-2.7 gh |Mac OS X 10.10|

- The record length data type is **short**
- Test your program by storing the data of 10 notebooks in a file and applying the five listed functionalities below.

The required functionalities:

- **Add** new record, always add new records in the end of file.
- **Delete** a record, just marks the record by * exactly after records length.
  Example ➔ the below record's length is 30, so its form will be like:
  30*----------
- **Update** a record given the **ID**, if the new record is the same size, then insert in the same place, if size is changed, perform **delete** then **add** operations.

- **Search** for a record given an **ID**, if found print its data, if not tell user that this device is not found. **Use sequential search.**
- **CompactFile** : due to the delete and the add operations, a lot of records' space should be reclaimed, so CompactFile function reclaims these spaces. After calling this function, the file should has no deleted records.

- **VisualizeFile**: this function prints a visual representation of a file.
  Example: if the file contains 10 records and we deleted the records have number 5,7 and 10, so the function's output will be like this:
  ----*-*--*

Example: Main Welcome Screen

```
                    Students Management System
    1)  Add a notebook
    2)  Update a notebook
    3)  Delete a notebook
    4)  Display a notebook
    5)  Compact the file
    6)  VisualizeFile
    7)  Exit
Please Enter Your Choice :
```